

Greedy GP Sensor Placement — Algorithm Explanations

Based on Krause, Singh & Guestrin (2008)
Near-Optimal Sensor Placements in Gaussian Processes
JMLR 9, 235–284

Contents

Background: Gaussian Processes and the Sensor Placement Problem	2
Gaussian Processes	2
Conditional Distribution	2
The Sensor Placement Problem	3
Helper / Utility Functions	3
Matrix Inversion: <code>mat_inv</code>	3
Triangular Solve: <code>solve_tri</code>	3
Quadratic Form: <code>quad_form</code>	3
Rank-1 Cholesky Downdate: <code>chol_downdate</code>	4
Greedy Maximum Entropy Selection	4
Criterion	4
Algorithm 1 (Greedy Max-Entropy)	5
<code>entropy_naive</code> — $O(ns^3 + s^4)$	5
<code>entropy_prec</code> — $O(ns^2)$	5
<code>entropy_prechol</code> — $O(ns^2)$	6
<code>entropy_chol</code> — $O(ns^2)$	6
Greedy Maximum Mutual Information Selection	7
Criterion	7
Algorithm 2 (Greedy Max-MI)	7
Schur Complement Update for the Precision (used in <code>mi_prec</code>)	7
<code>mi_naive</code> — $O(n^4s)$	8
<code>mi_prec</code> — $O(n^3)$	8
<code>mi_chol</code> — $O(n^3)$	8

Greedy Nystrom Ordering	9
Criterion	9
Greedy algorithm	9
nystrom_order_naive — $O(n^2 s^2)$	9
nystrom_order — $O(n^2 s)$	9
Summary Table	10
Kernel Interface	11

Background: Gaussian Processes and the Sensor Placement Problem

Gaussian Processes

A **Gaussian Process** (GP) is a probability distribution over functions. A zero-mean GP is fully characterised by its **kernel** (covariance function) $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, written

$$f \sim \mathcal{GP}(0, k).$$

For a finite set of candidate locations $V = \{x_1, \dots, x_n\} \subset \mathcal{X}$ the joint distribution is multivariate Gaussian:

$$f_V = (f(x_1), \dots, f(x_n))^\top \sim \mathcal{N}(0, K)$$

where the **kernel matrix** $K \in \mathbb{R}^{n \times n}$ has entries $K_{ij} = k(x_i, x_j)$.

Conditional Distribution

For any partition of V into a selected set S and its complement $\bar{S} = V \setminus S$, the GP conditional is also Gaussian:

$$f_{\bar{S}} \mid f_S \sim \mathcal{N}(K_{\bar{S}S} K_{SS}^{-1} f_S, K_{\bar{S}\bar{S}} - K_{\bar{S}S} K_{SS}^{-1} K_{S\bar{S}}).$$

The **conditional variance** of a single point $x_k \notin S$ is therefore

$$\sigma^2(x_k \mid X_S) = k(x_k, x_k) - k_{kS} K_{SS}^{-1} k_{Sk}$$

where $k_{kS} = [k(x_k, x_i)]_{x_i \in S}$ is the cross-covariance row vector.

The Sensor Placement Problem

Given a budget of s sensors to place among n candidate sites, we seek a set $S^* \subset V$, $|S^*| = s$ that maximises some information-theoretic criterion. The three criteria implemented here are:

Criterion	Objective
Maximum entropy	$\max_S H(X_S)$
Maximum mutual information	$\max_S I(X_S; X_{V \setminus S})$
Minimum Nystrom KL divergence	$\min_S \text{KL}(\text{GP}_{\text{full}} \parallel \text{GP}_{\text{Nystrom}}(S))$

All three are submodular functions of S , which means greedy selection provides a $(1 - 1/e)$ -approximation guarantee of the optimum.

Helper / Utility Functions

Matrix Inversion: `mat_inv`

For a symmetric positive definite (SPD) matrix M , the Cholesky factorisation $M = U^\top U$ (with U upper triangular) allows inversion as:

$$M^{-1} = U^{-1} (U^{-\top})$$

R's `chol2inv(chol(M))` implements this. The cost is $\approx n^3/3$ flops for the factorisation plus $\approx n^3/3$ for the inversion, roughly half the cost of a general LU-based inversion ($\approx n^3$ flops).

Triangular Solve: `solve_tri`

Solves $Lx = b$ (or $L^\top x = b$) for lower or upper triangular L . Triangular solves cost $O(n^2)$ and are the building block for all efficient kernel computations in this file.

Quadratic Form: `quad_form`

Computes the **quadratic form** $x_i^\top \Theta^{-1} x_i$ for each column x_i of a matrix $X \in \mathbb{R}^{p \times q}$.

Algorithm. Write $\Theta = U^\top U$ (upper Cholesky). Then:

$$x^\top \Theta^{-1} x = x^\top U^{-1} U^{-\top} x = \|U^{-\top} x\|^2$$

Solve $U^\top t = x$ by forward substitution, then return $\|t\|^2$. This is cheaper than explicitly forming Θ^{-1} first.

In the entropy algorithms this computes the **reduction in variance** from conditioning on the selected set:

$$\sigma^2(x_k \mid X_S) = k(x_k, x_k) - \underbrace{k_{kS} K_{SS}^{-1} k_{Sk}}_{\text{quad_form}(K_{SS}, k_{kS}^\top)}$$

Rank-1 Cholesky Downdate: `chol_downdate`

Given a lower Cholesky factor L of $A = LL^\top$, and a vector u , computes the lower Cholesky factor of the **rank-1 downdate**:

$$A - uu^\top = \tilde{L}\tilde{L}^\top$$

Algorithm: Givens rotations. Process columns $i = 1, \dots, n$ sequentially. At column i , the current diagonal entry L_{ii} and carry entry u_i define a 2D rotation that zeros out u_i while updating the new diagonal:

$$\begin{aligned} \tilde{L}_{ii} &= \sqrt{L_{ii}^2 - u_i^2} \\ c_1 &= \frac{L_{ii}}{\tilde{L}_{ii}}, \quad c_2 = \frac{u_i}{\tilde{L}_{ii}} \end{aligned}$$

Column i and the carry vector u are then updated:

$$\begin{aligned} L_{:,i} &\leftarrow c_1 L_{:,i} - c_2 u \\ u &\leftarrow \frac{u}{c_1} - \frac{c_2}{c_1} L_{:,i}^{\text{new}} \end{aligned}$$

After all n columns the result L is the Cholesky factor of $A - uu^\top$.

This operation is used in `mi_chol` to remove points from the precision Cholesky as they are added to the selected set S .

Greedy Maximum Entropy Selection

Criterion

The differential entropy of a multivariate Gaussian $X_S \sim \mathcal{N}(0, K_{SS})$ is:

$$H(X_S) = \frac{1}{2} \log \det K_{SS} + \frac{|S|}{2} (1 + \log 2\pi)$$

Greedy maximising $H(X_S)$ is equivalent to greedily maximising $\log \det K_{SS}$. By the chain rule of entropy, the greedy step reduces to:

$$x_k^* = \arg \max_{x_k \in V \setminus S} H(x_k \mid X_S) = \arg \max_{x_k \in V \setminus S} \sigma^2(x_k \mid X_S)$$

i.e. always add the point with the **largest conditional variance**.

Algorithm 1 (Greedy Max-Entropy)

Input: candidate set V , budget s
 $S \leftarrow \emptyset$
for $i = 1, \dots, s$:
 $k^* \leftarrow \arg \max_{k \in V \setminus S} \sigma^2(x_k \mid X_S)$
 $S \leftarrow S \cup \{k^*\}$
return S

The conditional variance update at each step uses:

$$\sigma^2(x_k \mid X_S) = K_{kk} - K_{kS} K_{SS}^{-1} K_{Sk}$$

The four implementations below differ only in **how they maintain** K_{SS}^{-1} (or an equivalent factorisation) across iterations.

entropy_naive — $O(ns^3 + s^4)$

At each of the s steps, the submatrix K_{SS} is passed to **quad_form**, which calls **chol()** from scratch. This costs $O(|S|^3)$ per iteration, giving total cost $O(ns^3 + s^4)$.

entropy_prec — $O(ns^2)$

Maintains the **precision matrix** $P = K_{SS}^{-1}$ and updates it by the **Schur complement block inversion formula** after adding each point.

When point k is added to S , the new joint covariance is:

$$K_{S \cup \{k\}, S \cup \{k\}} = \begin{pmatrix} K_{SS} & k_{Sk} \\ k_{kS} & K_{kk} \end{pmatrix}$$

By the block matrix inversion lemma, the new precision is:

$$P_{\text{new}} = \begin{pmatrix} P + w v v^\top & -w v \\ -w v^\top & w \end{pmatrix}$$

where

$$w = \frac{1}{\sigma^2(x_k \mid X_S)}, \quad v = P K_{Sk}.$$

The conditional variance update for all remaining points reduces to a **rank-1 subtraction**:

$$\sigma^2(x_j \mid X_{S \cup \{k\}}) = \sigma^2(x_j \mid X_S) - \frac{\text{Cov}^2(x_j, x_k \mid X_S)}{\sigma^2(x_k \mid X_S)}$$

where $\text{Cov}(x_j, x_k \mid X_S) = K_{jk} - K_{jS} K_{SS}^{-1} K_{Sk}$.

entropy_prechol — $O(ns^2)$

Instead of the precision matrix, maintains the lower Cholesky factor L of K_{SS} , where $K_{SS} = LL^\top$. Uses the “**up-looking**” Cholesky update: when the new point k is added, the new Cholesky factor satisfies

$$\begin{pmatrix} K_{SS} & k_{Sk} \\ k_{kS} & K_{kk} \end{pmatrix} = \begin{pmatrix} L & 0 \\ \ell^\top & d \end{pmatrix} \begin{pmatrix} L^\top & \ell \\ 0 & d \end{pmatrix}$$

Solving $L\ell = K_{Sk}$ (size $|S|$ forward substitution) gives ℓ , and the new diagonal entry is $d = \sqrt{K_{kk} - \|\ell\|^2}$.

entropy_chol — $O(ns^2)$

The most cache-friendly variant. Maintains an $n \times s$ matrix L where column i is the i -th column of the lower Cholesky factor of the **full** kernel matrix K , restricted to the selected set.

Left-looking Cholesky column update at step i :

$$L_{:,i} \leftarrow \frac{K_{:,k} - L_{:,1:i-1} L_{k,1:i-1}^\top}{\sqrt{L_{k,i}}}$$

The conditional variance update is simply:

$$\sigma^2(x_j \mid X_{S \cup \{k\}}) = \sigma^2(x_j \mid X_S) - L_{j,i}^2$$

Demonstration that all four variants agree:

```
# RBF (squared-exponential) kernel with lengthscale = 1
rbf <- function(X1, X2) {
  n1 <- nrow(X1); n2 <- nrow(X2)
  d2 <- as.matrix(dist(rbind(X1, X2)))[1:n1, (n1+1):(n1+n2), drop=FALSE]
  exp(-d2^2 / 2)
}
rbf_diag <- function(X) rep(1, nrow(X))

set.seed(42)
X <- matrix(rnorm(30), ncol = 2) # 15 points in 2D

data.frame(
  variant = c("entropy_naive", "entropy_prec", "entropy_prechol", "entropy_chol"),
  selected_indices = c(
    paste(entropy_naive(X, rbf, rbf_diag, 4), collapse = " "),
    paste(entropy_prec(X, rbf, rbf_diag, 4), collapse = " "),
    paste(entropy_prechol(X, rbf, rbf_diag, 4), collapse = " "),
    paste(entropy_chol(X, rbf, rbf_diag, 4), collapse = " ")
  )
)
```

```
##          variant selected_indices
## 1  entropy_naive      1 13 7 10
## 2  entropy_prec      1 13 7 10
## 3 entropy_prechol      1 13 7 10
## 4  entropy_chol       1 13 7 10
```

Greedy Maximum Mutual Information Selection

Criterion

Mutual information between the selected set X_S and the unselected set $X_{\bar{S}} = X_{V \setminus S}$ is:

$$I(X_S; X_{\bar{S}}) = H(X_S) + H(X_{\bar{S}}) - H(X_V) = \frac{1}{2} \log \frac{\det K_{SS} \cdot \det K_{\bar{S}\bar{S}}}{\det K}$$

For Gaussian variables, MI decomposes using the chain rule. Adding point k to S gives the greedy score:

$$\Delta_k = I(x_k; X_{V \setminus (S \cup \{k\})} \mid X_S) = H(x_k \mid X_S) - H(x_k \mid X_{V \setminus S})$$

For Gaussians entropy is a log:

$$\Delta_k = \frac{1}{2} \log \sigma^2(x_k \mid X_S) - \frac{1}{2} \log \sigma^2(x_k \mid X_{V \setminus S})$$

Maximising over k is equivalent to maximising the **product**:

$$x_k^* = \arg \max_{k \in V \setminus S} \underbrace{\sigma^2(x_k \mid X_S)}_{\text{cond_var1}[k]} \cdot \underbrace{\sigma^2(x_k \mid X_{V \setminus S})}_{\text{cond_var2}[k]}$$

Why is cond_var2 the diagonal of the precision?

The **full conditional** variance of x_k given all other points in $\bar{S}$ is related to the precision matrix $P_{\bar{S}} = K_{\bar{S}\bar{S}}^{-1}$ by:

$$\sigma^2(x_k \mid X_{\bar{S} \setminus \{k\}}) = \frac{1}{[K_{\bar{S}\bar{S}}^{-1}]_{kk}}$$

This is the **Schur complement** property: the k -th diagonal of the precision equals the reciprocal of the full conditional variance of the k -th variable.

Algorithm 2 (Greedy Max-MI)

```

Input:  $V, s$ 
 $S \leftarrow \emptyset, \quad P \leftarrow K^{-1}$ 
for  $i = 1, \dots, s$  :
     $k^* \leftarrow \arg \max_{k \in V \setminus S} \sigma^2(x_k \mid X_S) \cdot [P_{V \setminus S}]_{kk}^{-1}$ 
     $S \leftarrow S \cup \{k^*\}$ 
     $P_{V \setminus S} \leftarrow \text{remove } k^* \text{ from } P \text{ (Schur complement)}$ 
return  $S$ 
    
```

Schur Complement Update for the Precision (used in mi_prec)

When point k (at position j in the current candidate vector) is moved from $\bar{S}$ to S , the precision matrix of the remaining candidates updates as:

$$P_{\text{new}} = \left(P_{\bar{S} \setminus \{k\}, \bar{S} \setminus \{k\}} \right)^{-1 \leftarrow \text{this IS the update}} = P_{-j, -j} - \frac{P_{-j, j} P_{j, -j}}{P_{j, j}}$$

where $-j$ denotes all rows/columns except j . This corresponds to **marginalising** x_k out of the precision: conditioning in covariance is marginalising in precision, and vice versa.

mi_naive — $O(n^4 s)$

Recomputes both conditional variances from scratch every iteration. Each outer step costs $O(n^3)$ for inverting $K_{\bar{S}\bar{S}}$.

mi_prec — $O(n^3)$

Initialises $P = K^{-1}$ once ($O(n^3)$), then maintains $P_{\bar{S}}$ using the Schur complement deletion above ($O(n^2)$ per step). Also maintains a left-looking Cholesky for **cond_var1** (same as **entropy_chol**).

mi_chol — $O(n^3)$

Avoids storing the full $n \times n$ precision matrix by maintaining its lower Cholesky factor L_2 where $K^{-1} = L_2^\top L_2$.

Initialisation of L_2 .

Let J be the flip permutation matrix ($J_{ij} = 1$ if $i + j = n + 1$, 0 otherwise). Then $K_{\text{rev}} = J K J$ (the kernel matrix with rows and columns reversed). Compute $K_{\text{rev}} = U^\top U$ via Cholesky. The inverse of K can be obtained without explicit inversion:

$$K^{-1} = J(U^\top U)^{-1}J = JU^{-1}U^{-\top}J = (JU^{-\top}J)^\top(JU^{-\top}J)$$

Setting $L_2 = JU^{-\top}J$ (flip $U^{-\top}$ in both dimensions) gives a valid lower Cholesky of K^{-1} :

$$L_2^\top L_2 = K^{-1}$$

Diagonal entries give cond_var2:

$$[K^{-1}]_{kk} = \|L_2[k,]\|^2 = \text{rowSums}(L_2 * L_2)[k]$$

Downdate of L_2 .

When candidate k (at position j) is selected, the precision of the remaining candidates must be updated:

$$K_{\bar{S} \setminus \{k\}, \bar{S} \setminus \{k\}}^{-1} = K_{\bar{S}\bar{S}}^{-1} - \frac{K_{\bar{S}\bar{S}}^{-1} e_j e_j^\top K_{\bar{S}\bar{S}}^{-1}}{[K_{\bar{S}\bar{S}}^{-1}]_{jj}}$$

Defining $u = K_{\bar{S}\bar{S}}^{-1} e_j / \sqrt{[K_{\bar{S}\bar{S}}^{-1}]_{jj}}$ this is the rank-1 downdate $L_2 L_2^\top - uu^\top$, handled by **chol_downdate**.

Demonstration that all three MI variants agree:

```
data.frame(
  variant = c("mi_naive", "mi_prec", "mi_chol"),
  selected_indices = c(
    paste(mi_naive(X, rbf, rbf_diag, 4), collapse = " "),
    paste(mi_prec(X, rbf, rbf_diag, 4), collapse = " "),
    paste(mi_chol(X, rbf, rbf_diag, 4), collapse = " ")
  )
)
```



```
##      variant selected_indices
## 1 mi_naive      6 4 5 11
## 2 mi_prec       6 4 5 11
## 3 mi_chol       6 4 5 11
```

Greedy Nystrom Ordering

Criterion

The **Nystrom approximation** to a GP using inducing set S replaces the full posterior covariance with a low-rank approximation. The quality loss is measured by the **KL divergence**:

$$\text{KL}(\text{GP}_{\text{full}} \parallel \text{GP}_{\text{Nystrom}}(S)) = \frac{1}{2} \sum_{j \notin S} \log \frac{\sigma^2(x_j)}{\sigma^2(x_j \mid X_S)}$$

where $\sigma^2(x_j)$ is the marginal (prior) variance and $\sigma^2(x_j \mid X_S)$ is the posterior variance after conditioning on S .

This criterion is non-negative (since $\sigma^2(x_j \mid X_S) \leq \sigma^2(x_j)$) and equals zero when $S = V$.

Greedy algorithm

At each step, add the point k^* that **most reduces** the KL:

$$k^* = \arg \min_{k \in V \setminus S} \Delta \text{KL}(S, k) = \arg \min_k \sum_{\substack{j \notin S \\ j \neq k}} \log \frac{\sigma^2(x_j \mid X_S)}{\sigma^2(x_j \mid X_{S \cup \{k\}})}$$

The inner sum uses the fact that after adding k , the conditional variance of every remaining point updates as:

$$\sigma^2(x_j \mid X_{S \cup \{k\}}) = \sigma^2(x_j \mid X_S) - \frac{\text{Cov}^2(x_j, x_k \mid X_S)}{\sigma^2(x_k \mid X_S)}$$

nystrom_order_naive — $O(n^2 s^2)$

For each candidate k at each outer step, tentatively computes the full conditional variance update vector (cost $O(n)$ per candidate) and accumulates the log-ratio sum. This inner loop over n candidates costs $O(n^2)$ per outer step.

nystrom_order — $O(n^2 s)$

Maintains the full **conditional covariance matrix** $C = K - K_{:,S} K_{SS}^{-1} K_{S,:}$ (size reduces by one each step). All candidate scores can be computed simultaneously from a single matrix operation.

For candidate k (column/row j in the current C), the score matrix is:

$$\text{scores}[a, k] = \underbrace{C_{aa}}_{\sigma^2(x_a | X_S)} - \frac{C_{ak}^2}{C_{kk}}$$

The full KL score for candidate k is:

$$\Delta\text{KL}(k) = \sum_{a \neq k} \log \frac{C_{aa}}{\text{scores}[a, k]}$$

The diagonal is set to C_{aa} (a point does not affect its own variance), so the score matrix can be safely log-transformed and summed column-wise.

After selecting k (at position j), the conditional covariance shrinks via a rank-1 downdate:

$$C_{\text{new}} = C_{-j, -j} - u u^\top, \quad u = \frac{C_{:,j}}{\sqrt{C_{jj}}}$$

Demonstration that both Nystrom variants agree:

```
data.frame(
  variant = c("nystrom_order_naive", "nystrom_order"),
  selected_indices = c(
    paste(nystrom_order_naive(X, rbf, rbf_diag, 4), collapse = " "),
    paste(nystrom_order(X, rbf, rbf_diag, 4), collapse = " ")
  )
)

##           variant selected_indices
## 1 nystrom_order_naive      6 8 4 1
## 2       nystrom_order      6 8 4 1
```

Summary Table

Function	Criterion	Strategy	Complexity	Key data structure
entropy_naive	Max entropy	Recompute K_{SS}^{-1} each step	$O(ns^3 + s^4)$	$s \times s$ covariance submatrix
entropy_prec	Max entropy	Maintain precision K_{SS}^{-1}	$O(ns^2)$	$s \times s$ precision matrix
entropy_prechol	Max entropy	Maintain Cholesky of K_{SS}	$O(ns^2)$	$s \times s$ Cholesky (up-looking)
entropy_chol	Max entropy	Left-looking Cholesky on all n points	$O(ns^2)$	$n \times s$ Cholesky columns
mi_naive	Max MI	Recompute both precisions each step	$O(n^4 s)$	Full $n \times n$ kernel matrix

Function	Criterion	Strategy	Complexity	Key data structure
<code>mi_prec</code>	Max MI	Maintain precision of unselected	$O(n^3)$	$n \times n$ shrinking precision
<code>mi_chol</code>	Max MI	Maintain Cholesky of precision	$O(n^3)$	$n \times n$ shrinking Cholesky
<code>nystrom_order_naive</code>	Min Nystrom KL	Evaluate each candidate in a loop	$O(n^2 s^2)$	$n \times s$ Cholesky columns
<code>nystrom_order</code>	Min Nystrom KL	Vectorised score matrix + rank-1 update	$O(n^2 s)$	$n \times n$ conditional covariance

For practical use:

- When n is small and s is comparable to n , any variant works; prefer the naive variants for simplicity.
- For large n with small s , `entropy_chol` and `mi_chol` are the preferred choices.
- The Nystrom criterion explicitly optimises approximation quality rather than information, which can matter when the GP is used inside a downstream estimator.

Kernel Interface

All nine functions share the same two-argument kernel interface:

```
# kernel(X1, X2) : nrow(X1) × nrow(X2) covariance matrix
# kernel_diag(X) : length-nrow(X) marginal variance vector = diag(kernel(X, X))
```

`kernel_diag` exists for efficiency: for stationary kernels (RBF, Matérn) the diagonal is constant, so n evaluations suffice instead of the n^2 required by `diag(kernel(X, X))`. This is critical for maintaining the $O(ns^2)$ complexity of the efficient entropy variants.